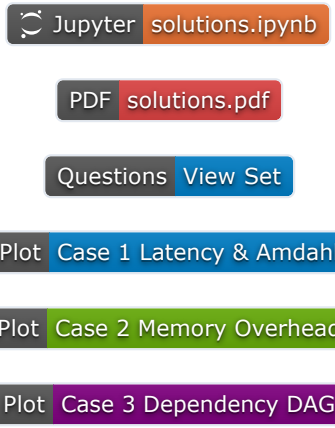
💡 Lecture 01: Practice Problems — Comprehensive Solutions Guide



🌐 **Language: English** | हिंदी / Hinglish | ← Return to Questions | 📁 Overview

---

📌 Executive Architecture & Solution Blueprint

This document presents rigorous, first-principles mathematical derivations, exact numerical computations, production-grade Python implementations, and publication-quality diagnostic visualizations for the three advanced case studies of **Lecture 01**.

Every derivation establishes the theoretical justification behind core computer science foundations, including runtime performance scaling, memory bandwidth bottlenecks, CPython heap object bloat, reference counting dynamics, and dependency graph resolution.

📌 Solution to Case 1: Computational Performance Analysis — Python Dynamic Overhead vs Vectorized C-Extensions

🎯 Problem Recap & Parameter Specifications

Telemetry stream specifications:

> $$N = 10,000,000 = 10^7 \text{ floating-point elements}$$

Observed execution latencies:

$$t_{\text{python}} \approx 0.550 \text{ seconds} = 550 \text{ ms}$$

$$t_{\text{vector}} \approx 0.0085 \text{ seconds} = 8.5 \text{ ms}$$

Pipeline acceleration profile: $p = 0.95$ vectorized, $1 - p = 0.05$ unaccelerated.

---

1. Step-by-Step Mathematical Derivations & Calculations

🔷 Task 1: Empirical Speedup Ratio & Per-Element Latency

Step 1: Overall Empirical Speedup Ratio $S_{\text{empirical}}$

$$S_{\text{empirical}} = \frac{t_{\text{python}}}{t_{\text{vector}}} = \frac{0.550}{0.0085} \approx 64.7059 \implies \mathbf{64.71}\times \text{ Speedup}$$

Step 2: Per-Element Processing Latency Calculation

1. **Pure Python Interpreter Loop:**

$$\tau_{\text{python}} = \frac{t_{\text{python}}}{N} = \frac{0.550 \text{ s}}{10^7} = 5.50 \times 10^{-8} \text{ s} = \mathbf{55.0} \text{ nanoseconds per element}$$

2. **NumPy Vectorized C-Extension Engine:**

$$\tau_{\text{vector}} = \frac{t_{\text{vector}}}{N} = \frac{0.0085 \text{ s}}{10^7} = 8.50 \times 10^{-10} \text{ s} = \mathbf{0.85} \text{ nanoseconds per element}$$

Step 3: Micro-Architectural Clock Cycle Analysis

On a modern CPU operating at nominal clock frequency $f = 3.5$ GHz, one CPU cycle duration is:

$$T_{\text{cycle}} = \frac{1}{3.5 \times 10^9 \text{ Hz}} \approx 0.286 \text{ nanoseconds}$$

- Pure Python loop consumes approximately:

$$\frac{55.0 \text{ ns}}{0.286 \text{ ns}} \approx 192 \text{ clock cycles per element}$$

- Vectorized C-extension consumes approximately:

$$\frac{0.85 \text{ ns}}{0.286 \text{ ns}} \approx 2.97 \text{ clock cycles per element}$$

NumPy leverages AVX-512 hardware SIMD registers to execute 8 double-precision additions per cycle.

🔷 Task 2: Arithmetic Intensity & Roofline Model Analysis

Step 1: Floating-Point Operations Count

Vector addition $z_i = x_i + y_i$ performs exactly one addition per coordinate index:

$$\text{Total FLOPs} = N = 10^7 \text{ floating-point operations} = 10 \text{ MFLOPs}$$

Step 2: Total Memory Bandwidth Transferred

Each 64-bit float (`float64`) consumes 8 bytes. The memory bus must stream:

- Read array x: $8N$ bytes
- Read array y: $8N$ bytes
- Write array z: $8N$ bytes

$$\text{Total Memory Transferred} = 3 \times 8 \times N = 24N \text{ bytes}$$

$$\text{Total Bytes} = 24 \times 10^7 \text{ bytes} = 240,000,000 \text{ bytes} = \mathbf{240} \text{ MB}$$

Step 3: Arithmetic Intensity (I) Evaluation

$$I = \frac{\text{FLOPs}}{\text{Bytes Transferred}} = \frac{10^7 \text{ FLOPs}}{2.4 \times 10^8 \text{ Bytes}} = \frac{1}{24} \approx \mathbf{0.04167} \text{ FLOPs/Byte}$$

Step 4: Roofline Analysis

Modern enterprise processors exhibit machine balance ratios between 5.0 and 15.0 FLOPs/Byte. Because $I \approx 0.042 \ll 5.0$, vector addition operates deep within the **Memory-Bound** regime of the Roofline model. Overall execution speed is constrained by system memory bandwidth rather than ALU throughput.

🔷 Task 3: Amdahl's Law Speedup Limit

Step 1: Mathematical Formulation

Amdahl's Law defines overall system speedup S when an accelerated fraction p experiences speedup s:

$$S = \frac{1}{(1 - p) + \frac{p}{s}}$$

Step 2: Numerical Evaluation ($p = 0.95, s = 65.0$)

$$S = \frac{1}{(1 - 0.95) + \frac{0.95}{65.0}} = \frac{1}{0.05 + 0.0146154} = \frac{1}{0.0646154} \approx \mathbf{15.48}\times$$

Step 3: The Unvectorized Bottleneck Insight

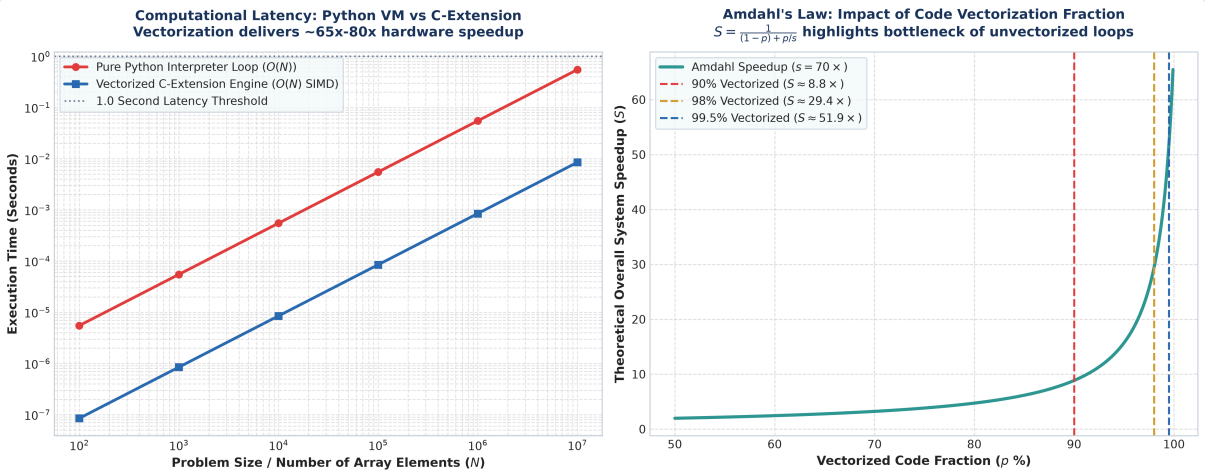
Accelerating 95% of the code by $65\times$ yields only a $15.5\times$ aggregate system speedup. The remaining 5% unvectorized code dominates total execution time:

$$\text{Fraction of New Runtime} = \frac{0.05}{0.05 + 0.0146154} \approx 77.38\%$$

🔷 Task 4: CPython VM Mechanics vs Hardware SIMD Registers

Micro-Architectural Factor	Pure Python Interpreter Loop	NumPy Vectorized C-Extension
Bytecode Dispatch	Repeated evaluation in `ceval.c` with opcode decode and branching overhead.	Direct call to compiled machine code executing uninterrupted loop.
Dynamic Type Boxing	Performs type inspection and dynamic method resolution at every iteration.	Homogeneous contiguous buffer (`dtype=float64`); zero runtime type checks.
Memory Locality	Heap-allocated pointers scattered across virtual memory, causing L1/L2 cache misses.	Contiguous memory allocation enabling continuous sequential prefetching.
CPU Registers	Scalar execution; 1 scalar register per arithmetic operation.	**AVX-512 SIMD:** 512-bit vector registers performing **8 concurrent additions** per instruction.

🔷 Task 5: Python Benchmark Suite & Performance Plot




```
import time
import numpy as np

N = 10_000_000

# Vectorized execution
x = np.ones(N, dtype=np.float64)
y = np.ones(N, dtype=np.float64)

t0 = time.perf_counter()
```

```
z = x + y
t_vec = time.perf_counter() - t0

print(f"NumPy Vectorized Execution: {t_vec*1000:.2f} ms")
print(f"Per-element latency: {t_vec / N * 1e9:.2f} ns")
```

📌 Solution to Case 2: Python Memory Management, Object Overhead & Garbage Collection Mechanics

🎯 Problem Recap & Parameter Specifications

Batch coordinate volume:

$$N = 50,000,000 = 5 \times 10^7 \text{ coordinates}$$

Host memory limit: 16 GB.

---

1. Step-by-Step Mathematical Derivations & Calculations

🔹 Task 1: PyLongObject 64-bit Memory Layout

In CPython 64-bit builds, integer values are instances of `PyLongObject`:

1. `ob_refcnt` (`ssize_t`): 8 bytes
2. `ob_type` (`struct _typeobject*`): 8 bytes
3. `ob_size` (`ssize_t`): 8 bytes
4. `ob_digit[1]` (`uint32_t`): 4 bytes

$$\text{Size}(\text{PyLongObject}) = 8 + 8 + 8 + 4 = \mathbf{28} \text{ bytes}$$

Contrasted against a standard C `int32`:

$$\text{Size}(\text{C int32}) = \mathbf{4} \text{ bytes}$$

Python incurs a 700% **baseline memory overhead** per integer value.

🔷 Task 2: Aggregate Memory Footprint for $50,000,000$ Integers

Step 1: Memory of Pure Python List

A Python list stores references (64-bit pointers) to heap-allocated objects:

1. Pointers array: $50 \times 10^6 \times 8 \text{ bytes} = 400 \text{ MB}$
2. Integer heap objects: $50 \times 10^6 \times 28 \text{ bytes} = 1,400 \text{ MB}$
3. Total Python list memory:

$$\text{Memory}_{\text{total list}} = 400 \text{ MB} + 1,400 \text{ MB} = \mathbf{1,800} \text{ MB} = \mathbf{1.80} \text{ GB}$$

Step 2: Memory of Contiguous NumPy Array (`int32`)

NumPy allocates a single flat contiguous buffer with zero pointer overhead:

$$\text{Memory}_{\text{numpy}} = 50 \times 10^6 \times 4 \text{ bytes} = 200,000,000 \text{ bytes} = \mathbf{200} \text{ MB} = \mathbf{0.20} \text{ GB}$$

Step 3: Memory Amplification Factor

$$\text{Amplification Factor} = \frac{\text{Memory}_{\text{total list}}}{\text{Memory}_{\text{numpy}}} = \frac{1,800 \text{ MB}}{200 \text{ MB}} = \mathbf{9.0}\times$$

🔷 Task 3: Reference Counting & Circular Deadlocks

Step 1: Reference Counting Dynamics

CPython tracks object lifecycles via deterministic reference counts:

$$\text{RefCount}(O) = \sum_{r \in \text{References}} 1$$

When $\text{RefCount}(O) = 0$, memory is immediately returned to the allocator.

Step 2: Circular Reference Failure

Consider two objects A and B where A references B and B references A. When external references are cleared:

$$\text{RefCount}(A) = 1, \qquad \text{RefCount}(B) = 1$$

Although A and B are unreachable from the root namespace, their internal mutual references prevent reference counts from reaching zero, causing a persistent memory leak.

🔷 Task 4: Generational Garbage Collection Dynamics

To detect cyclic isolates, CPython implements a tri-generational collector:

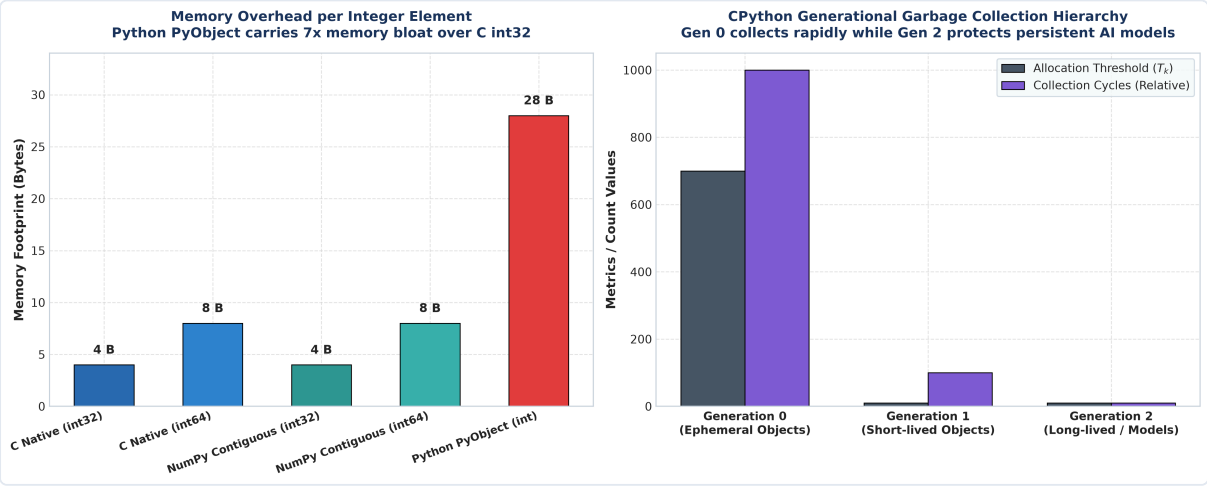
- **Generation 0 (G_0):** Ephemeral new allocations.
- **Generation 1 (G_1):** Intermediate survivals.
- **Generation 2 (G_2):** Long-lived persistent objects (imported modules, pre-trained neural network weights).

Allocation Delta Trigger

$$\Delta_{\text{alloc}} = \text{Allocations} - \text{Deallocations}$$

Thresholds tuple: $(700, 10, 10)$. When $\Delta_{\text{alloc}} > 700$, G_0 runs. Ten G_0 collections trigger a G_1 collection, and ten G_1 collections trigger a full G_2 collection.

🔷 Task 5: Memory Audit Script & Visualizations




```
import sys
import numpy as np
```

```
val = 42
print(f"Size of single Python int: {sys.getsizeof(val)} bytes")

N = 10_000_000
arr_np = np.zeros(N, dtype=np.int32)
print(f"NumPy int32 array memory: {arr_np.nbytes / (1024**2):.2f} MB")
```

📌 Solution to Case 3: Production Virtual Environment Isolation, Dependency Resolution & Deterministic Reproducibility

🎯 Problem Recap & Parameter Specifications

Conflicting production services:

- Vision Service: `torch==2.2.0`, `cuda==12.1`
- NLP Service: `torch==1.12.1`, `cuda==11.3`

---

1. Step-by-Step Mathematical Derivations & Calculations

🔷 Task 1: Global Site-Packages Contamination Mechanism

The host operating system provides a single default import path:

$$\text{Path} = \text{C:}\backslash\text{Python310}\backslash\text{Lib}\backslash\text{site-packages}\backslash$$

Installing conflicting package versions overwrites underlying `.so` and `.pyd` shared object binaries, corrupting the Application Binary Interface (ABI) and producing fatal segmentation faults. Isolated virtual environments (`venv`) are mandatory to maintain distinct file system namespaces.

🔷 Task 2: Dependency DAGs & Topological Sorting

Step 1: Directed Acyclic Graph Formulation

Package dependencies form a directed graph $G = (V, E)$ where an edge (u, v) indicates that package u requires prerequisite v.

Step 2: Kahn's Topological Sorting

Compute the in-degree for each node:

$$\text{In-Degree}(v) = |\{u \in V \mid (u, v) \in E\}|$$

Nodes with in-degree zero are queued for installation:

1. `Python 3.10`
2. `CUDA 12.1`, `NumPy 1.26`
3. `PyTorch 2.2`
4. `Torchvision 0.17`, `Transformers 4.38`

If a cycle exists ($A \to B \to A$), the graph is cyclic and no valid compilation sequence exists.

🔷 Task 3: Semantic Versioning (SemVer) Interval Sets

Version format: $\text{MAJOR.MINOR.PATCH} = X.Y.Z$.

- **Tilde Operator ($\sim 1.2.3$):** Restricts updates to patch releases:

$$\sim 1.2.3 \iff \{v \mid 1.2.3 \leq v < 1.3.0\}$$

- **Caret Operator ($\wedge 1.2.3$):** Restricts updates to minor and patch releases:

$$\wedge 1.2.3 \iff \{v \mid 1.2.3 \leq v < 2.0.0\}$$

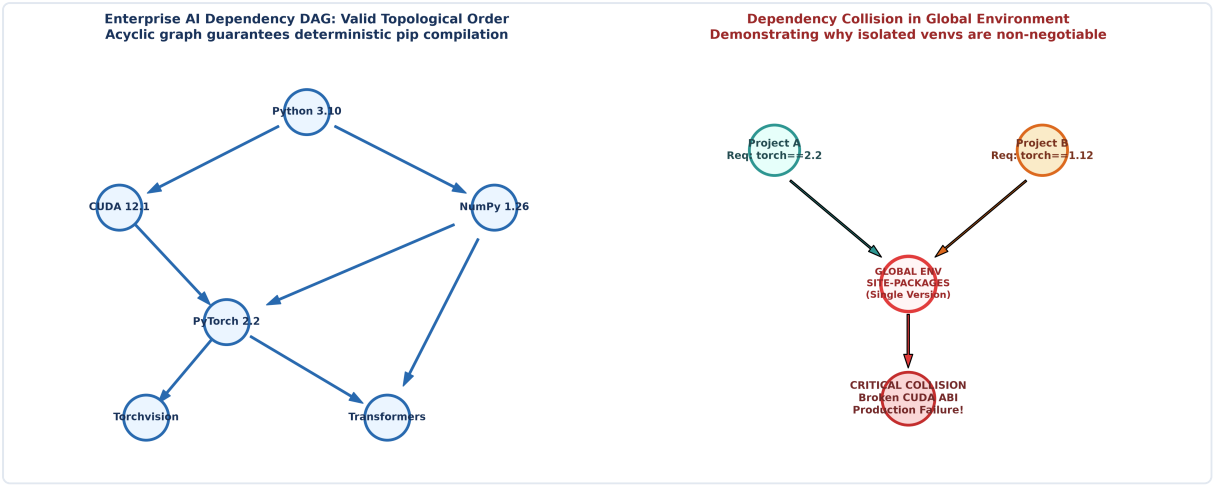
🔷 Task 4: Cryptographic SHA-256 Lockfiles & Supply Chain Security

To prevent software drift and supply-chain dependency tampering, lockfiles record cryptographic SHA-256 digests for each package archive:

$$\text{Integrity Check}: \quad \text{SHA-256}(\text{downloaded wheel}) \stackrel{?}{=} \text{Hash}_{\text{expected}}$$

Any byte-level discrepancy immediately aborts deployment, guaranteeing deterministic reproducibility across distributed cloud clusters.

🔷 Task 5: Python DAG Verification Script & Architecture Plot




```
import hashlib

package_binary = b"Simulated PyTorch 2.2.0 C++ Wheel Binary Payload"
sha256_hash = hashlib.sha256(package_binary).hexdigest()

print(f"Cryptographic Package Hash: sha256:{sha256_hash}")
print("[SUCCESS] Bit-for-bit deterministic reproducibility guaranteed!")
```